

Kleine Graphen Bibliothek

Im folgenden Text verwenden wir durchgehend das generische Femininum.

1.1. Ziel des Projektes

Gegenstand des Projektes ist die Implementierung einer kleinen Bibliothek zur Handhabung von Graphen verschiedenen „Typs“: gerichtete und ungerichtete Graphen, jeweils mit Kanten mit und ohne Gewichtung. Abgesehen von Datenstrukturen, die die verschiedenen Graphentypen darstellen, sollen noch einige im Kontext von Graphen häufig verwendete Algorithmen implementiert werden. Die Implementierung soll in der Programmiersprache C++ erfolgen.

Ein weiteres Lernziel dieses Projektes ist neben der Vertiefung Ihrer C++ Kenntnisse und der Umsetzung eines Softwareprojektes auch die Fähigkeit zu erwerben bzw. vertiefen, Ihre Arbeit kompakt schriftlich darzustellen.

1.2. Bestandteile

Im Folgenden finden Sie eine Übersicht über Bestandteile Ihrer Abgabe, die sich auf den zu schreibenden Code beziehen.

1.2.1. Code

BITTE BEACHTEN SIE

Es wird vorausgesetzt, dass **alle Gruppenmitglieder** sich in **vergleichbarem Umfang** am Schreiben des Quellcodes beteiligen. Das umfasst sowohl Library-Code als auch solchen für Tests und Beispiele zur Nutzung. Nutzen Sie Ihr Logbuch bzw. die Zusammenfassung des Logbuchs, um Ihre Beiträge zu dokumentieren. Fügen Sie in Ihrem Gruppen Git Repository ein (tabellarische) Übersicht bei, die die Beiträge der einzelnen Gruppenmitglieder aufschlüsselt.

Graphen Ihre Bibliothek soll die folgenden „Typen“ von Graphen abdecken:

- (i) Gerichtete Graphen mit *und* ohne gewichtete Kanten
- (ii) *Ungerichtete* Graphen mit *und* ohne gewichtete Kanten

Algorithmen Implementieren Sie für die entsprechenden Typen von Graphen eine Auswahl an häufig verwendete Algorithmen unter Berücksichtigung der implementierten Darstellungsform

des Graphentyps in Ihrem Code. Stellen Sie bitte sicher, dass Sie mit den Algorithmen *alle* Typen von Graphen abdecken; dokumentieren Sie bitte in Ihrem Git Repository (vgl. dazu auch Abschnitt 1.3.8), welche Algorithmen implementiert wurden und welche Graphentypen Sie damit abdecken.

Tests Schreiben Sie für *wesentliche* Bestandteile Ihrer Implementierungen Tests. Welches Testing Framework verwendet wird (z. B. Google Test[1], oder Catch2[2]), ist dabei Ihnen überlassen. Es sollte allerdings nachvollziehbar dokumentiert sein (vgl. Abschnitte 1.3.3 und 1.3.8) – und auch dargelegt werden! –, warum (oder auch warum nicht), Sie einen bestimmten Code-Bestandteil testen. Beispiele zum Schreiben von Tests finden Sie in den Hausaufgaben oder in der Dokumentation des gewählten Testing Frameworks.

Allgemein ist es empfehlenswert, sich *einfache* Testfälle zu überlegen, die dann ohne (allzu) großen Aufwand umgesetzt werden können. Zum einen wird dann das Schreiben des eigentlichen Tests einfacher, zum anderen ist der Test transparenter („Was genau wird hier geprüft?“) und auch selbst weniger fehleranfällig.

Beispiele Erstellen Sie für die wesentlichen Funktionalitäten Ihrer Bibliothek einfache Beispielprogramme zur Demonstration.

1.2.2. Build-System

Für die von Ihnen entwickelte Bibliothek sollte sich mit CMake ein „Build-System“ erstellen und damit fehlerlos kompilieren lassen (z. B. in eine statische Bibliothek `libGraph.a`). Ebenso sollten die Tests sowie auch die Beispiele (ggf. optional) kompilierbar sein; in beiden Fällen bietet es sich an, jeweils ausführbare Dateien zu erstellen.

Außerdem sollte es möglich sein, Ihre Bibliothek in ein anderes, ebenfalls auf CMake basierendes, Projekt einzubinden.

1.3. Prüfungsdokument

In diesem Abschnitt finden Sie nähere Informationen zu den erwarteten Inhalten Ihrer e-Portfolios. Eine kompakte Übersicht geben Ihnen Tabellen 1.1a und 1.1b; es wird aber dringend empfohlen, sich vor Beginn der Bearbeitung des Projektes mit den Inhalten der anderen Abschnitte vertraut zu machen.

1.3.1. Allgemeine Form und Bestandteile

Das Abgabeformat des Projektes (das „e-Portfolio“) besteht

- (a) für **jedes Gruppenmitglieds** aus einer **PDF Datei** mit den in Tabelle 1.1a aufgelisteten Bestandteilen, und
- (b) einem **Git Repository für die gesamte Gruppe** im JLU Gitlab, dessen Bestandteile in Tabelle 1.1b aufgelistet sind (vgl. auch Abschnitt 1.3.8).

TABELLE 1.1: Bestandteile des e-Portfolios.

(a) Bestandteile der PDF-Datei mit Reihenfolge und Zuordnung zu Gruppenmitgliedern sowie empfohlenem Seitenumfang.

POSITION	BESTANDTEIL	ENTHALTEN IM E-PORTFOLIO VON	UMFANG SEITEN
1	Titel	allen Gruppenmitgliedern	1
2	Logbuch	allen Gruppenmitgliedern	—
3	Zusammenfassung des Logbuchs	allen Gruppenmitgliedern	max. 4
4	Einschätzung der Gruppenmitglieder	allen Gruppenmitgliedern	1
5	Protokoll der Gruppentreffen	Sprecherin der Gruppe	1–2
6	Quellen und Hilfsmittel	allen Gruppenmitgliedern	—

(b) Bestandteile des Gruppen Git Repository.

POSITION	BESTANDTEIL
1	Quellcode der gesamten Gruppe (Implementation der Bibliothek, Beispiele und Tests) mit einer passenden Ordnerstruktur
2	Dokumentation des Gesamtprojektes
3	Dokumentation der Tests und Beispiele
4	Hierarchie an CMake-Konfigurationsdateien zum Konfigurieren des Buildsystems der Bibliothek, der Tests sowie der Beispiele

BITTE BEACHTEN SIE

Die in Tabelle 1.1a genannten Abschnitte (Bestandteile) müssen in Ihrem Abgabedokument visuell durch Überschriften, Abschnitte etc. und geeignete Formatierung klar erkennbar voneinander getrennt sein.

Dabei ist es unerheblich, welche Software Sie für die Erstellung (der Inhalte) der PDF-Datei verwenden (z. B. $\text{\LaTeX}$ oder ein Office-Programm), solange das PDF die geforderten Abschnitte und Inhalte hat und eine ansprechende äußere Form hat.

Das e-Portfolio hat gemäß den Modulbeschreibungen[3, 4] einen Umfang von **5–10 Seiten (ohne Programmcode)**. Die Seitenzahl fließt nicht in die Bewertung ein, dient aber als Richtwert. Halten Sie sich an den Grundsatz: **so lang wie nötig, so kurz wie möglich**. Eine Empfehlung zum Umfang der einzelnen Teile finden Sie in Tabelle 1.1a. Das **Logbuch** ist als Anhang beizufügen, **zählt nicht zum Seitenumfang**, wird aber als eigener Bestandteil bewertet.

Unabhängig vom konkreten Umfang gilt: Das e-Portfolio soll den individuellen Arbeitsprozess, die getroffenen Entscheidungen und den erreichten Zwischenstand verständlich und nachvollziehbar dokumentieren. Eine formal korrekte und gut strukturierte Darstellung ist Voraussetzung für eine angemessene Bewertung.

Achten Sie hierbei explizit darauf, Ihren Text durch Code-Listings (siehe auch Anhang A) – und ggf. auch Abbildungen, Tabellen oder Flußdiagramme – (allgemein „Visualisierungen“) zu untermauern und verweisen Sie bei Bedarf auf Ihren Programmcode. Visualisierungen wie eben beschrieben sind **keine** Screenshots sondern sollten auf eine sinnvolle Weise erstellt werden. Die folgende Auflistung nennt einige Möglichkeiten, Visualisierungen zu erstellen, *ohne* dabei den Anspruch auf Vollständigkeit zu erheben:

- Für Tabellen eignen sich die Tabellenfunktionen in Ihrer Textbearbeitungssoftware.
- Für Abbildungen können Sie geeignete Software wie z. B. Matplotlib[5] verwenden, um Abbildungen zu exportieren und in Ihr Dokument einzubinden. Ansonsten bietet natürlich auch Office-Software solche Funktionalitäten. Achten Sie hierbei auf eine ausreichende

Auflösung, eine sinnvolle Beschriftung und eine klare Darstellung der Daten.

- Code-Listings lassen sich in $\text{\LaTeX}$ beispielsweise mit dem Paket `listings`[6, 7] erzeugen. Eine andere Möglichkeit bietet Markdown in Kombination mit einer Software wie Quarto[8] oder Pandoc[9].¹

1.3.2. Titel

Die Titelseite dient der eindeutigen Zuordnung des e-Portfolios und enthält die folgenden Angaben:

- Thema
- Name
- Gruppennummer
- Übrige Gruppenmitglieder
- Sprecherin

Die Angaben sollen vollständig und übersichtlich dargestellt sein. Eine aufwändige Gestaltung ist nicht erforderlich; entscheidend ist, dass das Portfolio eindeutig zugeordnet werden kann.

1.3.3. Logbuch

Im Logbuch dokumentieren Sie Ihren **individuellen Arbeitsprozess** während des gesamten Bearbeitungszeitraums. Das Logbuch dient nicht der Darstellung eines „fertigen“ Ergebnisses, sondern der nachvollziehbaren Beschreibung Ihres Vorgehens. Ziel ist, dass der geleistete Arbeitsaufwand im erforderlichen Umfang plausibel wird und eigene Entscheidungen/Anpassungen im Code Entwicklungsprozess erkennbar sind.

BITTE BEACHTEN SIE

Das Logbuch ist dem e-Portfolio als **Anhang** beizufügen und wird nicht auf den Seitenumfang angerechnet. Gleichzeitig wird es als **eigener Bestandteil** bewertet und bildet die Grundlage für die Zusammenfassung des Logbuchs.

Die Einträge erfolgen **chronologisch**. Stichpunktartige Formulierungen sind ausdrücklich ausreichend, sofern der jeweilige Arbeitsschritt verständlich beschrieben wird. Wichtig ist, dass auch Entscheidungen, verworfene Ansätze, Schwierigkeiten und offene Fragen sichtbar werden; das kann beispielsweise eine Diskussion von verschiedenen Implementationsdetails oder Details zu bestimmten Tests umfassen.²

Jeder Logbucheintrag enthält mindestens

- das Datum des Eintrags,

¹Der Vollständigkeit halber sei erwähnt, dass sich mit [8, 9] auch komplette Dokumente auf der Basis von Markdown erstellen lassen. Das Erstellen von Tabellen bzw. das Einbinden von Bildern ist ebenfalls möglich.

²Es ist also durchaus denkbar, dass sie hier beispielsweise verschiedene Ansätze zur Implementation eines gewählten Algorithmus oder des Designs einer Klasse darstellen. Wenn Sie z. B. erkennen, dass eine vorherige Implementation durch die Verwendung von geeigneten Datenstrukturen aus der STL vereinfacht oder performanter wird, dann ist das Logbuch der richtige Ort, um die vorgenommenen Anpassungen zu dokumentieren. Der besseren Übersichtlichkeit halber bieten sich hier klickbare Verweise auf relevante frühere Einträge an. So vermeiden Sie die Wiederholung bereits erstellten Inhalts.

- einen kurzen, prägnanten Titel,
- eine knappe Zusammenfassung des jeweiligen Arbeitsschritts.

Verweisen Sie darauf, aus welchen Gruppentreffen (vgl. Abschnitt 1.3.6) die jeweiligen Arbeitsschritte hervorgehen. Geben Sie bei Abweichungen vom vereinbarten Vorgehen an, **wie** und **warum** Sie hier abgewichen sind.

Code-Listings (oder – falls zutreffend – Abbildungen und Tabellen) sollten zur Dokumentation des Arbeitsprozesses eingesetzt werden. Im Logbuch dienen diese nicht der Präsentation Ihrer Ergebnisse, sondern der Dokumentation Ihres Fortschrittes und müssen daher nicht den obigen Kriterien entsprechen. Das heißt insbesondere, dass Code-Listings auch größer sein können (und damit auch unübersichtlicher) oder Code einfach über einen Screenshot aus einem Texteditor dargestellt wird; sofern zutreffend, müssen auch Abbildungen nicht vollständig ausgearbeitet sein (z. B. bzgl. der Achsenbeschriftungen). Nichtsdestotrotz sollte auch hier immer erkennbar sein, was gezeigt wird. Dazu gehört auch, dass Code-Listings oder Abbildungen eine kurze Beschreibung enthalten (auch hier sind Stichpunkte vollkommen ausreichend) bzw. im Text referenziert werden; das erleichtert nachher bei der Zusammenfassung des Logbuchs das gezielte Einbinden von Inhalten aus dem Logbuch. Der Schwerpunkt des Logbuchs liegt hier auf der inhaltlichen Nachvollziehbarkeit des Vorgehens, nicht auf der Menge oder Ausarbeitung der Darstellungen. In Anhang A haben wir Ihnen ein kleines Beispiel für den Einsatz von Code-Listings zusammengestellt.

Im Logbuch soll erkennbar werden, wie die einzelnen Entwicklungsschritte aufeinander aufbauen; wichtig ist immer, dass Sie die getroffenen Entscheidungen und (sachlich) **begründen** und **nachvollziehbar darlegen** können.³

BITTE BEACHTEN SIE

Aus Ihrem Logbuch soll erkennbar sein, welche Teile der Gesamtimplementation Ihnen zuzuordnen sind; das gilt für Library-Code genauso wie für Tests und Beispiele zur Nutzung des Codes.

Das Logbuch bildet die Grundlage für die Zusammenfassung des Logbuchs. Eine sorgfältige, kontinuierliche und besonders **zeitnahe** Führung erleichtert daher sowohl die Zusammenfassung als auch die spätere Bewertung des Portfolios.

1.3.4. Zusammenfassung des Logbuchs

Die Zusammenfassung des Logbuchs enthält eine **ausformulierte** Darstellung Ihres bisherigen Vorgehens und Ihres erreichten Arbeitsstands auf Basis des Logbuch. Achten Sie dabei darauf, die darin dokumentierten Inhalte gezielt auszuwählen und sinnvoll zu gewichten.

Der Umfang der Zusammenfassung soll **maximal vier Seiten inklusive Code-Listings** (und ggf. Abbildungen/Tabellen) betragen. Der Fokus liegt auf einer prägnanten, inhaltlich dichten Darstellung.

In der Zusammenfassung soll deutlich werden,

- welche Fragestellungen/Aufgaben Sie im Kontext des Gesamtprojektes verfolgt/übernommen haben,

³Wenn z. B. eine Klasse viele **private** Methoden hat, drängt sich die Frage auf, warum diese Wahl getroffen wurde und warum diese anderen möglichen Lösungen vorgezogen wurde.

- welche Implementationsansätze Sie gewählt haben und warum,⁴ welche Tests Sie für wichtig erachten und wie sich Teile Ihrer Implementation sinnvoll demonstrieren lassen,
- welche Schwierigkeiten oder Grenzen aufgetreten sind,
- und welchen **Zwischenstand** Ihre Arbeit am Ende der Bearbeitungszeit erreicht hat.

Es ist ausdrücklich **nicht** erforderlich, ein vollständiges oder optimales Endergebnis zu präsentieren.⁵ Vielmehr soll nachvollziehbar werden, wie Sie mit dem Thema umgegangen sind und welche Erkenntnisse Sie aus Ihrer Arbeit gewonnen haben.

Zur Unterstützung der Darstellung sind geeignete Visualisierungen – insbesondere Code-Listings, Flußdiagramme aber ggf. auch Abbildungen und Tabellen – zu verwenden. Diese sollen gezielt eingesetzt werden und zur inhaltlichen Klarheit beitragen. Jegliche Abbildungen sind fortlaufend zu nummerieren, mit einer kurzen Beschreibung zu versehen und im Text zu referenzieren. Insbesondere, bei Code-Listings ist auf Kompaktheit zu achten; es soll kein überflüssiger Code gezeigt werden, sondern nur solche Teile, die zum besseren Verständnis ihrer Ausführungen geeignet sind (für ein Beispiel siehe Anhang A).

An geeigneten Stellen sollen klickbare Verweise auf relevante Einträge, Abbildungen oder Tabellen im Logbuch integriert werden, um detaillierte Informationen zugänglich zu machen, ohne den Fließtext der Zusammenfassung zu überfrachten.

1.3.5. Einschätzung der Gruppenmitglieder

In diesem Teil des e-Portfolios reflektieren Sie die Zusammenarbeit innerhalb Ihrer Gruppe. Ziel ist nicht, eine Bewertung der Leistung anderer Personen, sondern eine sachliche und faire Einschätzung der gemeinsamen Arbeitsweise.

Für jedes Gruppenmitglied geben Sie mindestens **eine Stärke und eine Schwäche** an, die sich aus der Zusammenarbeit im Projekt ergeben haben. Die Einschätzung soll nachvollziehbar begründet und respektvoll formuliert sein. Pauschale oder rein formelhafte Aussagen sind dabei wenig hilfreich. Der Fokus liegt auf der Reflexion von Arbeitsweisen, Kommunikation und Zusammenarbeit, nicht auf persönlichen Eigenschaften.

Als Richtwert: **2–4 Sätze pro Person** sind ausreichend, wenn eine Stärke und eine Schwäche konkret benannt und kurz begründet werden.

Die Angaben in diesem Abschnitt werden vertraulich behandelt und nicht an andere Gruppenmitglieder weitergegeben.

1.3.6. Protokoll der Gruppentreffen

Dieser Abschnitt ist nur im **e-Portfolio der Sprecherin** der Gruppe enthalten.

Im Protokoll der Gruppentreffen dokumentiert die Sprecherin den organisatorischen und inhalt-

⁴Das umfasst explizit auch getroffene Design-Entscheidungen wie z. B., welche Code-Bestandteile durch eine Klasse dargestellt werden, oder warum eine Funktion eine „freie Funktion“ und nicht eine Methode ist. Beachten Sie hierzu auch die Ausführungen in Bemerkungen zum Code. Es sollte deutlich werden, warum Sie bestimmte Entscheidungen bzgl. Ihrer Implementation getroffen haben (z. B. zur Vermeidung von Code-Duplikation im Sinne von DRY[10]).

⁵Es ist fast immer denkbar, dass Code noch performanter oder noch kompakter und wiederverwendbarer ist.

lichen Verlauf der Zusammenarbeit innerhalb der Gruppe. Ziel ist es, die gemeinsame Arbeitsplanung und deren Entwicklung über den Bearbeitungszeitraum nachvollziehbar darzustellen. Der Umfang dieses Abschnitts soll circa **ein bis zwei Seiten** betragen.

Für **jedes** Gruppentreffen sind mindestens anzugeben

- das Datum des Treffens,
- die teilnehmenden Gruppenmitglieder,
- eine kurze Zusammenfassung der besprochenen Inhalte.

Die Zusammenfassung kann stichpunktartig erfolgen. Wichtig ist, dass aus dem Protokoll hervorgeht,

- welche Arbeitsschritte vereinbart wurden,
- wie Aufgaben innerhalb der Gruppe verteilt wurden,
- und welche zeitlichen Absprachen getroffen wurden.

Im Verlauf der Bearbeitung soll zudem erkennbar werden, ob und wie sich die Planung im Laufe des Projekts verändert hat. Anpassungen des Vorgehens sind ausdrücklich zulässig und sollen dokumentiert werden, sofern sie aus neuen Erkenntnissen oder Rahmenbedingungen resultieren. Die Anzahl der Gruppentreffen ist Ihrer Gruppe überlassen. Achten Sie bitte außerdem darauf, die Dokumentation der einzelnen Gruppentreffen visuell klar voneinander zu trennen. Weiterhin soll in Ihren Logbüchern (vgl. Abschnitt 1.3.3) auf das entsprechende Gruppentreffen verwiesen werden, wenn Sie sich einem darin abgesprochenen Arbeitsschritt widmen oder davon abweichen.

1.3.7. Quellen und Hilfsmittel

Listen Sie hier die verwendeten Hilfsmittel auf, z. B.

- Webseiten (mit Zugriffsdatum!),
- Bücher,
- wissenschaftliche Artikel,
- Einsatz generativer KI (zumindest die Prompts angeben). Bitte sehen Sie davon ab, sich dabei C++ Quellcode jeglicher Art für Ihr Projekt generieren zu lassen.

BITTE BEACHTEN SIE

Verwendete Quellen sollen im Text in geeigneter Weise und an *geeigneter Stelle* kenntlich gemacht werden. Bitte verwenden Sie dafür einen einheitlichen Zitierstil (bspw. IEEE, Harvard, APA, Chicago). Möglichkeiten um Quellen zu verwalten bzw. einzubinden gibt es u. a. für L^AT_EX^a und Quarto^b; auch Office-Software bietet passende Möglichkeiten.

^a11.

^b12.

1.3.8. Gruppen Git Repository

Richten Sie für die Gruppe im Gitlab der JLU Giessen ein Repository ein, auf das alle Gruppenmitglieder Zugriff erhalten. Denkbar ist dabei z. B., dass die Sprecherin der Gruppe das Repository erstellt und darauf dann allen anderen Gruppenmitgliedern Zugriff ermöglicht; eine

sinnvolle Rolle für die eingeladenen Gruppenmitglieder ist dabei „Maintainer“ (siehe dazu auch [13]).

BITTE BEACHTEN SIE

- Stellen Sie beim Einrichten die Sichtbarkeit des Repositories auf **Private**. Dies beschränkt den Zugriff auf die Leute, denen explizit Zugriff gewährt wurde. Wenn Sie Ihr Projekt abgeben, fügen Sie bitte auch den **Dozenten (Marcel Giar)** mit der Rolle eines „Reporter“ zur Liste der zugriffsberechtigten Personen hinzu.
- Die Sprecherin vermerkt im PDF-Teil Ihres e-Portfolios an geeigneter Stelle die URL zum Gruppen Git Repository.
- Vermeiden Sie es bitte, dieses Dokument mit der Projektbeschreibung im Git Repository zu speichern.
- Stellen Sie bitte sicher, dass am Ende der Hauptbranch **main** alle von ihnen für die finale Bewertung vorgesehenen Inhalte enthält. Nur Inhalte dieses Branches werden für die Bewertung herangezogen; über verschiedene Branches „verstreute“ Inhalte werden nicht zur Bewertung herangezogen.

Die im Git Repository erwarteten Inhalte sind in Tabelle 1.1b aufgelistet.

Ziel ist es, dass jedes Gruppenmitglied die ihr zugewiesenen Inhalte in das Git Repository einpflegt. Gerade zu Beginn empfiehlt es sich, dafür verschiedene Branches zu erstellen (z.B. für jedes Gruppenmitglied einen) und zunächst nur individuell auf diesen Branches zu arbeiten, sodass Sie sich nicht gegenseitig stören. Nach einem gemeinsamen Review-Prozess – z. B. im Rahmen eines Gruppentreffens – können einzelne Branches zusammengeführt („gemergt“) werden. Stellen Sie bitte sicher, dass am Ende der **Hauptbranch main** alle von ihnen für die finale Bewertung vorgesehenen Inhalte enthält.⁶ Nur Inhalte dieses Branches werden für die Bewertung herangezogen; über verschiedene Branches „verstreute“ Inhalte werden nicht zur Bewertung herangezogen.

Die Inhalte des Repositories wie etwa C++-Code, CMake-Konfigurationsdateien, **README.md**-Dateien mit Dokumentation usw. sollen mit einer **sinnvollen Struktur** abgelegt werden. Dazu gehört auch eine entsprechende Ordnerstruktur mit jeweils „sprechenden“ Namen haben.⁷

Vermeiden Sie es bitte unbedingt, einfach nur die Inhalte der einzelnen Gruppenmitglieder ohne erkennbare Gesamtstruktur auf dem Hauptbranch abzulegen. Bemühen Sie sich stattdessen dem Projekt – bestehend aus der Gesamtheit der Einzelbeiträge – eine gemeinsame Struktur zu geben.

Im Quellcode soll erkennbar sein, **wer** zu diesem beigetragen hat; dabei ist es durchaus denkbar, dass mehrere Personen zu einer bestimmten Funktion, Klasse etc. beitragen. Bitte vermerken Sie in diesem Fall einfach alle zutreffenden Namen in einem Kommentar über der Funktion.

Neben dem Quellcode soll Ihre Repository auch **Dokumentation** zur Nutzung Ihres Projektes enthalten. Platzieren Sie diese in sinnvoller Weise in dem von Ihnen gewählten Ordner-Layout; in Gitlab eignen sich dafür besonders **README.md** mit Markdown-Inhalt. Ein solche Dokumentation kann z. B. darin bestehen aufzuzeigen, wie Ihre Bibliothek kompiliert oder in einem anderen Projekt verwendet werden kann. Weiterhin sollen die von ihnen erstellten Tests und Beispiele zur Nutzung der verschiedenen Funktionalitäten dokumentiert sein.⁸

⁶Prinzipiell ist im Rahmen eines individuellen Logbuchs ein Verweis auf den Inhalt eines Branch eines einzelnen Gruppenmitglieds denkbar.

⁷Um einen Eindruck über mögliche Strukturen einiger C++-Projekte zu erhalten, sei hier auf [2, 14, 15] als *mögliche* Referenzen verwiesen.

⁸Die Dokumentation sollte dabei so weit gehen, dass man bereits *ohne* einen Blick in den Quellcode der Tests zu

Bemerkungen zum Code Die folgenden Hinweise beziehen sich auf den von Ihnen geschriebenen Quellcode, dessen Strukturierung im Git Repository aber auch auf dessen Dokumentation in Ihren individuellen Logbüchern (siehe Abschnitt 1.3.3) bzw. deren Zusammenfassung (siehe Abschnitt 1.3.4).

- Verteilen Sie Ihren Code in sinnvoller Weise über Header- (typischerweise `.h` Suffix) und Quellcode-Dateien (typischerweise `.cpp` Suffix). Auch kann eine Gliederung einzelner Bestandteile in Unterordnern für die allgemeine Strukturierung des Projektes hilfreich sein. Bedenken Sie, dass bei der Verwendung von Funktions- und Klassentemplates der *gesamte* Code der Funktion/Klasse in einer Header-Datei vorhanden sein muss. Bei *Funktions-Templates* umfasst das die komplette Definition, bei *Klassen-Templates* die Klassen-Definition *sowie* die Implementation der Methoden.
- Verwenden Sie „modernes C++“ wo möglich. Die „Standard-Template-Library“ bietet Ihnen ein Vielzahl von Datenstrukturen (`std::vector` etc.) sowie Algorithmen (mit Komplexitätsgarantien!), die in vielen Fällen sehr hilfreich sind.
- Machen Sie sich Gedanken über das *Software-Design* Ihrer Bibliothek. Dies sollte v.a. im Hinblick auf Richtlinien wie „Don’t repeat yourself“ (DRY)[10] oder des „Single Responsibility Principle“ (SRP)[16, 17] erfolgen. Zudem können Sie die Testbarkeit einzelner Komponenten in diese Betrachtungen miteinbeziehen (Stellen Sie sich die Frage: „Kann ich Komponente X der Software sinnvoll testen?“ Wenn Sie feststellen, dass dies nicht ohne Weiteres möglich ist – oder nur unter größerem Aufwand – kann dies ein Zeichen für „mangelhaftes“ Software Design sein.). **Dokumentieren Sie Design-Entscheidungen in angemessenem Umfang in Ihrem Logbuch und/oder Ihrer Zusammenfassung des Logbuchs.**
- Machen Sie sich Gedanken darüber, welche Teile Ihres Codes sich sinnvoll als Klasse (oder auch als Klassenhierarchie) abbilden lassen, und welche ggf. besser freie Funktionen sein sollten (also keine an eine Klasse oder Objekt gebundenen Funktionen (Methoden)).
- Dokumentieren Sie als Gruppe im Git Repository gut sichtbar in einer `README.md` Datei welche Algorithmen implementiert wurden und welche Graphentypen Sie damit abdecken.

1.4. Bearbeitung

Sie arbeiten an dem Thema in einer **Gruppe aus zwei bis drei Personen**; eine Bearbeitung alleine ist **nicht** möglich. Es sind nur Gruppen bestehend aus **Studierenden des gleichen Studiengangs** zulässig. Vereinbaren Sie zeitnah einen ersten Termin für ein Gruppentreffen und bestimmen Sie zunächst eine Sprecherin. Diese Person dokumentiert die Gruppentreffen (vgl. Abschnitt 1.3.6) und gibt diese mit ihrem Logbuch ab (vgl. auch Tabelle 1.1a).

Das Projekt ist mit Absicht frei formuliert und Sinn ist es, dass Sie sich in einer vorgegebenen Bearbeitungszeit sinnvoll mit dem Thema auseinandersetzen. Sofern anwendbar, liegt es bei Ihnen die Inhalte entsprechend zu gewichten und die Arbeit sinnvoll unter allen Personen innerhalb Ihrer Gruppe aufzuteilen.

werfen (grob) weiß, was getestet wird. Für Details kann der Quellcode natürlich immer noch in Augenschein genommen werden. Das gilt sinngemäß ebenfalls für die Beispiele zur Nutzung der Bibliothek.

1.5. Abgabe des Portfolios

Jedes Gruppenmitglied muss ihr e-Portfolio abgeben. Achten Sie darauf, dass bei den individuellen Kategorien (siehe auch Allgemeine Form und Bestandteile) und der Titelseite alle Informationen vorhanden sind und dass die Sprecherin zusätzlich die Gruppentreffen (vgl. Abschnitt 1.3.6) dokumentiert hat.

Anschließend exportieren Sie Ihr Portfolio im **PDF-Format** (Abschnitt 1.3.1) und benennen es nach dem folgenden Schema:

OOP_<Gruppennummer>_<Nachname>_Portfolio.pdf

Ersetzen Sie dabei die „<Gruppennummer>“ durch Ihre Gruppennummer und „<Nachname>“ durch Ihren Nachnamen. Laden Sie anschließend diese **PDF-Datei** in Stud.IP in den vorgesehenen Abgabeordner hoch.

1.6. Bearbeitungszeit

Der **Bearbeitungszeitraum** beträgt **15 Wochen**. [3, 4] Das Datum für die Abgabe wird gesondert bekannt gegeben. Natürlich ist eine frühere Abgabe möglich. Sobald die notwendigen Dateien in Stud.IP hochgeladen sind, gilt das Projekt als final bearbeitet.

Die reine Bearbeitungszeit – also der Workload – beträgt 3 CP bzw. umgerechnet 90 (15 + 75) Zeitstunden pro Gruppenmitglied. [3, 4] Hierbei handelt es sich um eine typische Bearbeitungszeit.

1.7. Bewertung

Die folgenden Tabellen schlüsseln die Punkte auf die einzelnen Bestandteile Ihrer e-Portfolios auf. Dabei gibt Ihnen Tabelle 1.2 eine Gesamtübersicht; in dieser Tabelle sind auch klickbare Links zu den anderen Tabellen enthalten.

TEILPUNKTE UND ABSTUFUNG

Eine Abstufung der auf ein Kriterium vergebenen Punkte erfolgt in Schritten von einem Punkt. Bei Bedarf können auch halbe Punkte vergeben werden. Die kleinstmögliche Punktzahl pro Kriterium ist immer 0.

TABELLE 1.2: Gesamtübersicht der Bewertung einzelner Bestandteile des **e-Portfolios** mit Gewichtung für die Gesamtbewertung.

BESTANDTEIL	BEWERTUNGSART	GEWICHT	MAX. PUNKTE
Titel & Allgemeine Form und Bestandteile	individuell	2,5%	1
Logbuch	individuell	10%	4
Zusammenfassung des Logbuchs	individuell	40%	6 + 4 + 6 = 16
Einschätzung der Gruppenmitglieder	individuell	5%	2
Protokoll der Gruppentreffen	Gruppe	10%	4
Quellen und Hilfsmittel	individuell	7,5%	3
Gruppen Git Repository	Gruppe	25%	10
Summe		100%	40

TABELLE 1.3: Bewertung Titel und Allgemeine Form und Bestandteile

PUNKTE	KRITERIUM
max. 1 Punkt	Titel und Allgemeine Form
1	Portfolio ist eindeutig zuordenbar (Name, Gruppennummer, Thema), alle geforderten Bestandteile sind vorhanden und klar voneinander getrennt. Das Dokument ist insgesamt gut lesbar und formal bewertbar.
0	Mindestens eine dieser Voraussetzungen ist nicht erfüllt (z. B. fehlende Angaben, unklare Struktur, formale Mängel, die die Bewertung erschweren).
Maximal: 1	

TABELLE 1.4: Bewertung Logbuch

PUNKTE	KRITERIUM
max. 2 Punkte (additiv)	Grundlagen
1	Eigene Beiträge: Die Eigenbeiträge zur Gesamtimplementation sind klar zu denen anderer Gruppenmitglieder abgegrenzt.
1	Substanz und Transfer: Einträge dokumentieren tatsächliche Bearbeitungsschritte und es entsteht der Eindruck einer tiefergehenden Auseinandersetzung mit der Thematik.
max. 2 Punkte (gestuft)	Nachvollziehbarkeit
2	Der Arbeitsprozess ist überzeugend nachvollziehbar: Zwischenschritte sowie Entscheidungen/Änderungen sind dokumentiert, nachvollziehbar begründet und eigene Ideen und Ansätze sind klar erkennbar. Der Fortschritt im Gesamtverlauf des Projektes wird ersichtlich.
1	Der Arbeitsprozess ist nachvollziehbar und enthält eigene Entscheidungen/Anpassungen; Sinn und Zweck einzelner Schritte allerdings nur begrenzt erkennbar oder nur teilweise ausgearbeitet.
0	Der Arbeitsprozess ist kaum nachvollziehbar oder bleibt überwiegend pauschal.
Maximal: 4	

TABELLE 1.5: Bewertung der Zusammenfassung des Logbuchs

(a) Grundlagen und Arbeitsprozess

PUNKTE	KRITERIUM
max. 3 Punkte (additiv)	Grundlagen
1	Aufgaben und Ziel im Kontext des Projekts klar erkennbar.
1	Zwischenstand am Ende der Bearbeitungszeit klar beschrieben.
1	Zentrale Implementationsergebnisse (Library-Code, Tests und Beispiele) verständlich benannt.
max. 3 Punkte (gestuft)	Arbeitsprozess und Methodeneinsatz:
3	Wesentliche Implementationsansätze klar benannt und begründet. Wesentliche Design-Entscheidungen ausreichend diskutiert und deren Zweck klar benannt. Relevanz von Tests für wichtige Code-Bestandteile und deren Testbarkeit ausreichend diskutiert.
2	Wesentliche Implementationsansätze benannt und begründet; Design-Entscheidungen und Relevanz von Tests grundsätzlich nachvollziehbar, Darstellung aber nicht durchgehend ausgearbeitet.
1	Wesentliche Implementationsansätze, Design-Entscheidungen und Relevanz von Tests nur knapp dargestellt und begründet, wodurch allgemeine Nachvollziehbarkeit eingeschränkt ist.
0	Gewähltes Vorgehen bzgl. Implementation, Tests etc. unbegründet und wirkt willkürlich.
Maximal: 6	

(b) Reflexion und Einordnung

PUNKTE	KRITERIUM
max. 2 Punkte (gestuft)	Grenzen/Unsicherheiten:
2	Wesentliche Grenzen/Unsicherheiten konkret benannt und passend eingeordnet.
1	Grenzen/Unsicherheiten genannt, aber eher allgemein oder knapp.
0	Keine kritische Reflexion erkennbar.
max. 2 Punkte (gestuft)	Schlussfolgerungen/Ausblick:
2	Schlussfolgerungen und nächster Schritt plausibel und konkret.
1	Einordnung/Ausblick vorhanden, aber vage oder wenig konkret.
0	Keine Schlussfolgerungen/kein Ausblick erkennbar.
Maximal: 4	

(c) Visualisierungen

PUNKTE	KRITERIUM
max. 3 Punkte (additiv)	Code-Listings/Abbildungen/Tabellen: formale Mindestqualität
1	Lesbarkeit & Auflösung (keine Screenshots).
1	Beschriftung & Legende/Kodierung eindeutig und konsistent.
1	Einbettung im Text (Caption/Referenz + Take-away).
max. 3 Punkte (gestuft)	Code-Listings/Abbildungen/Tabellen: Aussagekraft & Angemessenheit
3	Sehr passend; nicht irreführend; stützt Argumentation/Entscheidungen klar. Insbesondere die eingesetzten Code-Listings stützen die Ausführungen bzgl. getroffener Design-Entscheidungen.
2	Insgesamt passend; kleinere Schwächen, z. B. durch Code-Listings mit (teils) irrelevantem Inhalt.
1	Nur bedingt passend; Aussagekraft begrenzt/Interpretation dünn.
0	Fehlend oder kaum verwertbar/unklar/irreführend.
Maximal: 6	

TABELLE 1.6: Bewertung Einschätzung der Gruppenmitglieder

PUNKTE	KRITERIUM
max. 2 Punkte (gestuft)	Einschätzung der Gruppenmitglieder:
2	Für alle Gruppenmitglieder je mindestens eine Stärke und eine Schwäche konkret benannt und kurz begründet; respektvoll formuliert; Fokus auf Arbeitsweise/Kommunikation/Zusammenarbeit.
1	Grundsätzlich vorhanden, aber unvollständig oder überwiegend allgemein/pauschal; Begründungen eher knapp, geringe Differenzierung.
0	Fehlend oder nicht verwertbar.
Maximal: 2	

TABELLE 1.7: Bewertung Protokoll der Gruppentreffen

PUNKTE	KRITERIUM
max. 4 Punkte (additiv)	Protokoll der Gruppentreffen
1	Gruppentreffen sind dokumentiert (Datum, Teilnehmende, kurze Zusammenfassung der Inhalte).
1	Arbeitsplan/folgende Arbeitsschritte sind festgehalten (Aufgaben, Zuständigkeiten, grobe Zeitschiene).
1	Änderungen/Abweichungen im Verlauf sind erkennbar (Anpassungen nachvollziehbar).
1	Protokolle sind insgesamt konsistent und für die Arbeitsorganisation erkennbar hilfreich.
Maximal: 4	

TABELLE 1.8: Bewertung Quellen und Hilfsmittel.

PUNKTE	KRITERIUM
max. 3 Punkte (additiv)	Quellen und Hilfsmittel
1	Auffindbar und prüfbar: Quellen/Hilfsmittel sind eindeutig identifizierbar, z. B. URL mit Zugriffsdatum oder DOI; Software mit Name und Version (ggf. mit Link/Repository).
1	Im Text kenntlich gemacht: Externe Inhalte werden im Text nachvollziehbar referenziert und es wird ein einheitlicher Zitierstil verwendet.
1	Transparenz bei KI: Einsatz generativer KI ist nachvollziehbar dokumentiert.
Maximal: 3	

TABELLE 1.9: Bewertung des Gruppen Git Repository.

PUNKTE	KRITERIUM
max. 2 Punkte (gestuft)	Formale Struktur
2	Projekt kann mit CMake konfiguriert und kompiliert werden; eine Einbindung in als externes CMake Projekt ist leicht möglich. Ordnerstruktur und Layout des Projektes sind übersichtlich und verständlich und das Projekt wird als Einheit präsentiert.
1	Teils Mängel bei CMake (z. B. Konfiguration/Kompilieren funktioniert teilweise nicht), Verwendbarkeit als externes Projekt (lässt sich nicht einbinden, oder nur mit Aufwand), Ordnerstruktur und Layout (z. B. Layout teils irreführend oder teils uneinheitliches Gesamtprojekt).
0	CMake-Konfiguration nicht funktionsfähig (auch nicht als externes Projekt einbindbar), gänzlich unstrukturiertes Layout.
max. 8 Punkte (additiv)	Dokumentation und Auswahl
max. 2 Punkte (gestuft)	Gesamtprojekt: Zuordnung von Code zu Gruppenmitgliedern, Gesamtübersicht, vorhandene Funktionalitäten, Verwendbarkeit des Codes
max. 3 Punkte (gestuft)	Tests: Dokumentation, vorhandene Tests („test coverage“), sinnvoll ausgewählt, informativ, Ausführung
max. 3 Punkte (gestuft)	Beispiele: Dokumentation, Umfang, sinnvoll ausgewählt, informativ (Ausgabe, Kommentare im Code), Ausführung
Maximal: 10	

Referenzen

- [1] GoogleTest developers. *GoogleTest*. Version 1.17.0. 30. Apr. 2025. URL: <https://github.com/google/googletest>.
- [2] Phil Nash, Chris Thrasher und Martin Hořeňovský. *Catch2*. Version 3.12.0. 28. Dez. 2025. URL: <https://github.com/catchorg/Catch2>.
- [3] *Spezielle Ordnung für den Bachelorstudiengang Angewandte Informatik*. Justus-Liebig-Universität Gießen. 2023. URL: https://www.uni-giessen.de/de/mug/7/pdf/7_35/07/7_35_07_8/7_35_07_8_1ae (besucht am 02.02.2026).
- [4] *Spezielle Ordnung für den Masterstudiengang Data Science*. Justus-Liebig-Universität Gießen. 2023. URL: https://www.uni-giessen.de/de/mug/7/pdf/7_36/07/7_36_07_8/7_36_07_8_1ae (besucht am 02.02.2026).
- [5] J. D. Hunter. “Matplotlib: A 2D graphics environment”. In: *Computing in Science & Engineering* 9.3 (2007), S. 90–95. DOI: 10.1109/MCSE.2007.55.
- [6] Jobst Hoffmann, Carsten Heinz und Brooks Moses. *listings – Typeset source code listings using L^AT_EX*. 2025. URL: <https://ctan.org/pkg/listings> (besucht am 02.02.2026).
- [7] Jobst Hoffmann, Carsten Heinz und Brooks Moses. *The Listings Package*. Version 1.11b. 14. Nov. 2025. URL: <https://mirror.physik.tu-berlin.de/pub/CTAN/macros/latex/contrib/listings/listings.pdf> (besucht am 02.02.2026).
- [8] J.J. Allaire u. a. *Quarto*. Jan. 2022. DOI: 10.5281/zenodo.5960048. URL: <https://doi.org/10.5281/zenodo.5960048>.
- [9] John MacFarlane, Albert Krewinkel und Jesse Rosenthal. *Pandoc*. Version 3.8.3. 1. Dez. 2025. URL: <https://github.com/jgm/pandoc>.
- [10] Andrew Hunt und David Thomas. *The Pragmatic Programmer*. Addison-Wesley, 1999. ISBN: 0-201-61622-X.
- [11] Philip Kime, Moritz Wemheuer und Philipp Lehman. *The bibl_{at}ex Package*. Version 3.21. 10. Juli 2025. URL: <https://ctan.project-creative.net/macros/latex/contrib/bibl_{at}ex/doc/bibl_{at}ex.pdf> (besucht am 06.02.2026).
- [12] J.J. Allaire u. a. *Citations*. quarto.org. URL: <https://quarto.org/docs/authoring/citations.html> (besucht am 06.02.2026).
- [13] JLU Gitlab maintainers. *Roles and permissions*. URL: <https://gitlab.ub.uni-giessen.de/help/user/permissions.md> (besucht am 02.02.2026).
- [14] Eigen developers. *Eigen*. Version 5.0.1. 11. Nov. 2025. URL: <https://gitlab.com/libeigen/eigen/>.
- [15] LightGBM developers. *Light Gradient Boosting Machine*. Version 4.6.0. 15. Feb. 2025. URL: <https://github.com/microsoft/LightGBM>.
- [16] Robert C. Martin. *The Principles of OOD*. butunclebob.com. 11. Mai 2005. URL: <http://www.butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod> (besucht am 03.02.2026).
- [17] Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. Pearson, 2002. ISBN: 0-13-597444-5.
- [18] Klaus Iglberger u. a. “Expression Templates Revisited: A Performance Analysis of Current Methodologies”. In: *SIAM Journal on Scientific Computing* 34.2 (Jan. 2012), S. C42–C69. ISSN: 1064-8275, 1095-7197. DOI: 10.1137/110830125. URL: <http://epubs.siam.org/doi/10.1137/110830125>.

- [19] Klaus Iglberger u. a. “High performance smart expression template math libraries”. In: *2012 International Conference on High Performance Computing & Simulation (HPCS)*. 2012 International Conference on High Performance Computing & Simulation (HPCS). Madrid, Spain: IEEE, Juli 2012, S. 367–373. ISBN: 978-1-4673-2362-8 978-1-4673-2359-8 978-1-4673-2361-1. DOI: 10.1109/HPCSim.2012.6266939. URL: <http://ieeexplore.ieee.org/document/6266939/>.
- [20] Johan Mabilie. *How we wrote xtensor*. medium.com. 24. Nov. 2019. URL: <https://johan-mabilie.medium.com/how-we-wrote-xtensor-9365952372d9> (besucht am 07.02.2026).
- [21] *Comparison of linear algebra libraries*. Wikipedia, The Free Encyclopedia, Wikimedia Foundation. 25. Okt. 2026. URL: https://en.wikipedia.org/wiki/Comparison_of_linear_algebra_libraries (besucht am 07.02.2026).
- [22] *std::floating_point*. cppreference.com. URL: https://en.cppreference.com/w/cpp/concepts/floating_point (besucht am 05.02.2026).

A. Beispiel für die Verwendung von Code-Listings

Hier soll Ihnen ein kurzes Beispiel für den Einsatz von Code-Listings gezeigt werden. Dabei ist es unerheblich, wie Sie diese erstellen – insbesondere mit welcher Software. Vielmehr soll aufgezeigt werden, wie Sie diese gezielt einsetzen können.

Im Folgenden wollen wir kurz annehmen, dass es unsere Aufgabe ist, für eine Klasse, die einen Vektor aus der Mathematik repräsentiert, die Operation der komponentenweise Addition zu implementieren.⁹ Eine kurze Diskussion der gezeigten Code-Listings könnte dann folgendermaßen aussehen:

BEMERKUNGEN

Wichtig sind in Code-Listings generell Zeilennummern; diese erleichtern allgemein die Orientierung und erlauben es zudem auf einfache Weise, noch einmal gesondert auf einen bestimmten Bereich des Listings zu verweisen. Syntax-Highlighting ist ebenfalls empfehlenswert. Wie bei Abbildungen und Tabellen auch empfiehlt sich eine kurze Beschreibung direkt am Code-Listing; diese sollte „für sich selbst stehen“ können, also insbesondere unabhängig vom Fließtext sein und den Inhalt des Code-Listings kurz und prägnant beschreiben.

LISTING A.1: Deklaration des für die Addition relevanten Methoden-Templates der `MathVector` Klasse. Es wird das `std::floating_point` concept^[22] verwendet, um die Templateparameter auf Gleitkommazahltypen wie `float` oder `double` zu beschränken. Diese Methode bezeichnen wir als „in-place“ Addition, da sie das Objekt direkt verändert.

```
1  template < std::floating_point T >
2  class MathVector
3  {
4  public:
5      using self_type = MathVector< T >;
6      /*...*/
7      template < std::floating_point U >
8      self_type &operator+=( MathVector< U > const &rhs );
9      /*...*/
10 };
```

BEMERKUNGEN

Beachten Sie, dass wir an dieser Stelle in Listing A.1 nur die für die Diskussion wesentlichen Teile des Code zeigen. Die eigentliche Implementation der Methode für die Addition mit einem anderen Objekt vom Typ `MathVector< U >` ist an dieser Stelle für das Verständnis unerheblich. Vielmehr möchten wir hier betonen, dass wir ein Methoden-Template verwenden, um auch Objekte mit verschiedenem Elementtyp addieren zu können. Der gezeigte Code ist für diese Aussage vollkommen ausreichend.

Listing A.1 zeigt einen Ausschnitt aus der Definition der `MathVector` Klasse,¹⁰ insbesondere die Deklaration des Operators `operator+=` für die Addition mit einem anderen `MathVector` Objekt („in-place“ Addition, das Objekt hier selbst verändert wird). Ein Aufruf dieser Methode für

⁹Wesentlich weitergehende Diskussionen zu Implementation und Design in diesem Kontext existieren *beispielsweise* (keine vollständige Liste) zu Bibliotheken für lineare Algebra^[18, 19], und darin enthaltene Referenzen^[20] bzw. für mehrdimensionale Arrays^[20], u. a. Abschnitte 5 und 6]. Eine ausführlichere Liste zu Bibliotheken zu linearer Algebra (C++ und andere Sprachen) findet sich in ^[21].

¹⁰Streng genommen handelt es sich um ein Klassen-Template. Der Einfachheit halber sei dies aber immer als implizit angenommen.

LISTING A.2: Deklaration eines Funktions-Templates mit `std::floating_point` concept[22] in beiden Templateparametern für die Addition von `MathVector` Objekten mit verschiedenem Elementtyp sowie ein beispielhafter Aufruf der Funktion. Diese freie Funktion gibt als Ergebnis ein neues `MathVector` Objekt zurück, dessen Elementtyp durch den der linken (`lhs`) und rechten (`rhs`) Seite der binären Additionsoperation bestimmt ist (über `std::common_type_t`).

```

1  template < std::floating_point T1, std::floating_point T2 >
2  MathVector< std::common_type_t< T1, T2 > >
3  operator+( MathVector< T1 > const &lhs,
4             MathVector< T2 > const &rhs ); // template of a free function
5
6  int main()
7  {
8      auto const vl{ MathVector( 3, 1.0 ) }, vr{ MathVector( 3, 2.0f ) };
9      auto const vres{ vl + vr }; // calls operator+
10     /*...*/
11 }

```

zwei `MathVector` Objekte `v` und `w` sieht dann z. B. so aus: `v += w`. Wie auch für die Klasse wird für die Methode ebenfalls ein Template verwendet (vgl. Zeile 7), damit auch die Addition von zwei Objekten mit unterschiedlichem Elementtyp möglich ist – etwa für `MathVector< double >` und `MathVector< float >`.

BEMERKUNGEN

Wie auch schon bei Listing A.1 haben wir uns in Listing A.2 nur auf den für die Diskussion wirklich notwendigen Code beschränkt. Code-Listings – gerade für Klassen – können schnell recht lang werden, was diese dann im Endeffekt unübersichtlich und für Ihre Diskussion nicht zweckdienlich macht.

Um Ausdrücke wie `vr + vl` verwenden zu können (siehe Zeile 9 in Listing A.2) – wobei beide Variablen `MathVector` Objekte mit einem bestimmten Elementtyp sind –, müssen wir noch einen binären Operator der Addition bereitstellen. Hier bietet es sich an, diesen als *freie Funktion* zu implementieren (vgl. Listing A.2). Zwar ist es auch möglich, diesen als Methode in die `MathVector` Klasse zu integrieren, allerdings ist es im Gegensatz zum `operator+=` nicht zwingend erforderlich. Im Sinne eines „schlanken“ Klassen-Interfaces soll die Klasse *nicht* mit nicht unbedingt notwendigen Methoden „überladen“ werden. Weitere Operatoren für arithmetische Operationen (etwa die Addition eines `MathVector` Objektes mit einem Skalar) lassen sich außerdem so in bequemer Weise hinzufügen.¹¹ Falls bevorzugt haben wir so auch die Möglichkeit, die Definition von `MathVector` und freien Funktionen, die damit arbeiten, in *verschiedenen* Header-Dateien zu trennen; u. U. kann dies für eine verbesserte Struktur des Gesamtprojektes sorgen.

BEMERKUNGEN

Im vorangehenden Textblock haben wir einige Male auf Listing A.2 verwiesen (hier kann auch auf einzelne Zeilen verwiesen werden). Solche Querbezüge – diese sollten i. d. R. klickbar sein! – sind für eine Diskussion unerlässlich und erlauben es überhaupt erst, das Listing (oder allgemein auch andere Formen der Visualisierung) einzubinden und die damit verbundenen Aussagen zu tätigen. Visualisierungen, die nicht in den Fließtext durch Querbezüge eingebunden werden, sind normalerweise überflüssig.

¹¹Voraussetzung ist dafür aber oft die Existenz der passenden in-place Methode (z. B. `operator+=`) innerhalb der `MathVector` Klasse. Gleichmaßen ist es dann aber auch vorstellbar, dass – falls man diese Operationen *alle* zu Methoden der `MathVector` Klasse macht (also `operator*=`, `operator*`, usw.) – das Interface derselben sehr groß und damit auch unübersichtlich wird.

Wie auch schon bei der `operator+=` Methode verwenden wir in Listing A.2 ein Funktions-Template für die freie Funktion `operator+`. Dies erlaubt uns wiederum flexibel `MathVector` Objekte verschiedenen Elementtyps zu addieren und erhalten ein `MathVector` Objekt mit einem passenden Elementtyp, `MathVector< std::common_type_t< T1 , T2 > >`.
